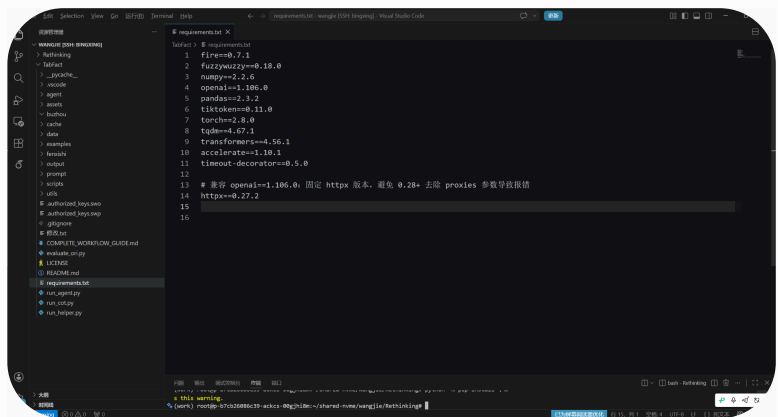




1987806d-c35d-4de6-b...
PDF

我想给codex的输入指令是：

我之前第一份工作完整做完了，并整理成第一篇论文也已经中稿了。现在开始搞第二份工作，第二份工作的思路是：论文思路：

从“语法查错”到“语义逻辑审计” (Semantic Logic Auditing)

1. 核心动机：提出并解决“执行信任偏差” (Execution-Trust Bias)

当模型调用外部工具时，往往会产生一种新的错觉：“代码跑通了 = 答案是对的”。Python 解释器只能发现语法错误 (Syntax Error) 或运行时错误 (Runtime Error)。如果模型提取了错误的列，或者把“求平均”写成了“求总和”，代码依然会完美运行并返回一个错误数字。这就是“执行信任偏差”。

2. 创新机制：代码与意图的“逆向一致性验证” (Reverse Consistency Verification)

你的 Analyst Agent 不能仅仅处理 Error Log，它必须在代码执行成功后进行更深层的介入：

代码逆向翻译：当 Reasoner Agent 生成了一段 Python 代码并成功执行出结果后，Analyst Agent 的任务不是去看结果对不对，而是阅读这段代码，并将其反向翻译成自然语言

的推理步骤。

意图对齐冲突检测：Analyst 将“反向翻译出的推理步骤”与“原始问题 (Question)”进行对比审计。如果发现代码的实际逻辑（比如过滤掉了某个月份）与原始问题的意图不符，Analyst 就会抛出“语义逻辑异常 (Semantic Logic Exception)”。

学术价值：你将软件工程中的“逻辑 Bug 审查”引入了 Agent 的反思机制中，这比处理解释器抛出的 `KeyError` 要高级得多。它延续了你第一份工作中“打破验证偏差”的核心思想，形成了一个完整的学术故事。

论文故事线 (Storyline) 建议

“针对复杂表格问答，现有基于工具的 Agent 往往受困于‘逻辑静默失败（代码跑通但逻辑错）’和‘工具死锁（在错误工具中无限报错）’。本文提出了一种动态多模态代理架构：不仅通过 Analyst Agent 进行深度的‘代码-意图语义审计’以防止静默失败，还引入了动态路由机制，在工具调用失效时自适应回退到文本推理，实现了精准度与鲁棒性的双重提升。”

EHRAgent: Code Empowers Large Language Models for Few-shot Complex Tabular Reasoning on Electronic Health Records (EMNLP 2024)

ReAcTable 的做法：它使用的是单智能体 (Single-Agent) 的 ReAct 范式。模型生成代码后交给执行器，如果报错，它只做非常底层的、硬编码的异常处理：比如 SQL 找不到列名，它就尝试去之前的中间表里找；如果 Python 提示缺包，它就帮你 `pip install`。如果遇到它处理不了的其他异常，它就直接强迫模型输出最终答案（"force the LLM to generate an answer"）。

你的降维打击（方向1：语义逻辑审计）：ReAcTable 完全陷入了我们刚才聊到的“执行信任偏差 (Execution-Trust Bias)”。只要代码跑通了，哪怕逻辑是错的（比如题目要求求和，代码写了求平均），ReAcTable 就会毫无察觉地把错误的中间表喂给下一轮。而你的 Analyst Agent 会进行“代码逆向翻译”，去审查跑通的代码是否符合初始问题的真实语义。这是从“语法查错”到“逻辑审计”的质变。

“Python 执行成功之后的语义逻辑审计”。也就是说，真正的新问题不是“让 LLM 会写 Python”，而是：代码跑通以

后，系统仍然可能因为错误列、错误过滤条件、错误聚合方式而产生更隐蔽的幻觉。

从文本推理验证偏差，扩展到工具增强推理中的执行信任偏差。

创新点必须落在：Python 代码执行成功以后，如何发现代码语义和原问题意图不一致。不是只判断程序对不对，而是把程序逆向解析成多粒度意图结构，再检查哪一层与问题不一致。审计代码是否忠实实现了原始问题意图。

结构化意图对齐：

第一层（粗粒度）：终态意图对齐 (Global Intent Alignment)。

检查最终返回值是否符合问题期望。

例子：

问题问“Which country...?”，预期答案类型是实体；

代码返回 `df['score'].sum()`，实际是数值；

直接触发 Global Type Mismatch。

这一层可以检测 answer type、answer cardinality、是否需要 yes/no、是否需要排名实体、是否需要数值差值等。

第二层（中粒度）：操作逻辑对齐 (Operator Logic Alignment)。

操作：将代码（如一段 Pandas 脚本）解析为操作序列。比如 `df.groupby('Year').sum()` 解析为“按年份分组求和”。分析师对比原始问题，检查是否遗漏了重要的操作（比如题目要求“大于2010年”，但解析的步骤中没有“筛选 (Filter)”操作）。或者问题要求 average，代码用了 sum；或者问题要求 highest，代码只求了总和，这一层就能拦截。

第三层（细粒度）：参数与单元格对齐 (Parameter & Entity Alignment)。

检查具体列名、实体值、常数、比较符、时间条件。

操作：检查解析出来的具体参数（列名、常数）。比如原始问题说的是“East Region”，代码解析出来发现用的是 `region == 'Eastern'`，在此处进行细粒度的实体匹配审查。

需要判断这是同义映射、表内别名，还是错误实体。

再比如：

问题：after 2010；

代码：year >= 2010；

这里是细粒度比较符错误，因为 after 通常对应 >，不是 >=。

这一层最好不要完全交给 LLM 判断，应该结合表格里的真实 cell value、列名归一化、字符串相似度、日期/数值解析来做。

Question Intent IR:

```
{
  answer_type: "number",
  target_column: "sales",
  filters: [
    {"column": "region", "op": "=", "value": "East"},
    {"column": "year", "op": ">", "value": 2022}
  ],
  aggregation: "sum"
}
```

Code Intent IR:

```
{
  return_type: "number",
  used_columns: ["region", "year", "sales"],
  filters: [
    {"column": "region", "op": "=", "value": "East"}
  ],
  aggregation: "sum"
}
```

然后审计器发现：

Semantic Logic Exception:

missing_filter: year > 2022

level: operator / parameter

你可以让 LLM 参与 IR 生成，但最好加上程序分析部分。例如：

用 Python AST 提取最后返回值；

用 Pandas 操作 trace 记录 filter / groupby / agg / sort / merge；

用表格 schema 做列名和 cell value grounding；

用 LLM 只处理难以规则化的语义匹配，比如 “Eastern” 是否等于 “East Region”。

我的思路主要体现在改动在采用Python时候的处理方式，但是大部分问题采用文本推理解决比较好，所以最后是采用联合推理（例如文本跑5次，Python跑5次，然后用多数投票得到最终答案？）取得效果比较好吗？还是说只考虑llm用Python解决tableqa。然后对比的时候和别人的方法也是只比较用Python解决table qa？

然后一定要有多粒度的思想，这个是我老师强调要加进去的。

我第一篇论文的所有代码在本地：D:\master 1\CS\VS code project\rethinking\wangjie 下面（注意这个文件夹下面所有的代码都不允许改变，只能从此处的代码复制）

第二篇论文我复制了第一篇论文一点可能用到的代码，然后在vscode软件里面连接了Linux服务器。你先查看第一篇论文的所有文件，如果用可以在第二篇论文用到的，可以复制后在Linux服务器粘贴。Linux服务器代码环境已经创建了一点，环境名字为work，需要安装环境时候用清华镜像。只允许在这个Linux服务器处更改所有文件。

工作二，我想先采用调用api的方式，调用示例：

```
import openai
client = openai.OpenAI(api_key="<$API_KEY>",
base_url="https://$BASE_URL/v1/")
response = client.chat.completions.create(model="$模型ID", #
model to send to the proxy messages=[ {
"role": "user", "content": "Hello world" } ]
) print(response)
```

模型ID：DeepSeek-V3.2，网址：<https://ai.paratera.com/>
apikey：sk-LNEI7ost0NuzdeFtzePucg（注意apikey不能泄露）

我要求跑wtq和tabfact测试集时候，先用DeepSeek-V3.2跑从0到第30条数据，然后你改进后的方法，再用用DeepSeek-V3.2跑从0到第30条数据，查看准确率是否有大大提升，然后你根据跑出来的日志和结果文件，查看推理步骤，并反复修改代码文件，最终得到你最满意的效果，才能停止此次codex项目。

vscode软件打开了两个窗口，一个是第一篇论文的完整项目，一个是第二篇论文连接服务器后的项目。再次强调，只允许增删改第二篇论文连接服务器后的项目。

额外补充：（下面内容和输入指令无关，你自己可以根据咱俩的对话知识然后思考）

我也把第一篇论文上传让你更好的了解我的领域，你也要思考借鉴当前他人的优势或者其他领域的可以用在表格问答上的方法，能不能更好的优化整体思路，数据集和第一篇论文一致。

收起

思考了 3m 16s

可以。你这个第二篇工作的主线已经比较清楚了，但我建议把**“论文创新”和“最终系统为了刷效果采用的工程策略”**分开，否则很容易最后变成“多跑几次 + 投票”，反而把真正有价值的创新冲淡。

你第一篇论文已经把 Reasoner、独立 Analyst、Arbiter 三者分开，通过外部审计打破 self-verification bias。
1987806d-c35d-4de6-bad7-d10e6d4... 更关键的是，你第一篇论文在 Limitations 里已经明确写了：当前 Analyst 还没有引入 Python/SQL 等外部符号执行器，并把 **tool-assisted verification** 列为后续重要方向。
1987806d-c35d-4de6-bad7-d10e6d4... 所以第二篇从“文本推理的验证偏差”自然延伸到“工具增强推理的执行信任偏差”，故事是接得上

我建议你把核心创新锁死在这里

不是“LLM 会不会写 Python”，也不是“Python 报错以后怎么修”，而是：

Python 已经成功运行，系统凭什么相信它忠实实现了问题意图？

把整个机制正式化成：

Question → Question Intent IR → Code → Code Intent IR → Multi-Granularity Semantic Alignment → Semantic Logic Exception → Targeted Repair

其中老师强调的“多粒度”不要只写在 prompt 里，而要真正成为方法结构：

- **L1 Global Intent Alignment**：答案类型、基数、任务类型、最终返回对象是否一致。
- **L2 Operator Logic Alignment**：filter / aggregation / comparison / ranking / groupby / arithmetic / temporal operations 是否遗漏、错误或多余。
- **L3 Parameter & Entity Alignment**：列、实体、常数、比较符、年份、单元格、别名等是否真正对齐。

最好让每层都有独立输出、独立置信度、独立异常类型。这样将来做 Global only → Global+Operator → Full 3-level 的消融实验，就可以直接证明“多粒度”本身有价值，而不只是一个概念包装。

不建议“文本5次 + Python5次然后多数投票”

主要原因不是效果一定不好，而是**论文故事会变差**。

SynTQA 已经发现符号方法与文本方法具有互补优势：Text-to-SQL 更擅长算术和长表，而端到端文本 TQA 在模糊问题、非标准 schema 等情况下更有优势，并通过选择器融合两类结果。  ACL Anthology H-STAR 2025 更已经直接做了**自适应 textual/symbolic reasoning**，按问题类型选择语义或符号推理。  ACL Anthology 所以到了你的第二篇，“**动态路由文本还是代码**”本身已经很难作为主创新。

而且 5+5 有三个问题：API 成本大约直接乘十；最终提升无法说明是 Semantic Audit 带来的还是简单 self-consistency 带来的；如果 Python 审计成功修出了正确答案，却被五个错误文本答案多数投票压掉，反而会损伤你的核心方法。

我更建议最终系统做成：

简单/语义型问题 → Text Reasoning

数值、筛选、排序、多行聚合等 → Python Reasoning + Semantic Audit

Router 不确定或两路风险都高 → Text + Audited Python 双路，再交给 Arbiter

也就是说，大部分问题不必强迫 Python，但凡是进入 **Python** 路径，都必须经过你的三层语义审计。

而且 routing 建议作为**增强模块**，不要一开始就做。先证明：

Vanilla Python Agent → + Semantic Audit

显著提升，然后再做：

Adaptive Text/Python → + Semantic Audit

这样论文因果关系最清楚。

对比实验也不要只跟 Python 方法比

最好分成两个层次。

机制层面对比一定要公平控制变量：

DeepSeek-V3.2 + Python

vs.

DeepSeek-V3.2 + Python + Multi-Granularity Semantic Audit

模型、prompt 基础设置、采样次数、Python executor 尽量一致。这样才能证明你的核心贡献。

最终系统层面则还应该跟 TableQA 总体 SOTA / strong baselines 比，而不是只挑 Python 方法。因为 MACT 已经做 planning agent + coding agent + tool use,  ACL Anthology TIDE

已经通过结构化 triples 辅助 decomposition 和 verification,  OpenReview EHRAgent 也已经利用代码执行及 execution feedback 来迭代改进代码。  ACL Anthology 这些都会成为审稿人自然想到的相关工作。

你真正需要强调的是：这些工作的工具执行主要帮助生成、执行、错误恢复或结构验证；你的研究问题进一步问“代码成功执行后，如何验证其执行语义是否忠实于原始自然语言意图”。写论文时最好逐篇全文核查后再下“现有方法没有……”这种强结论，不要只根据摘要直接绝对化。

甚至可以借一点软件工程/形式化验证领域的思想：2025 年已有工作尝试先把自然语言需求形成形式化 intent/specification，再验证生成代码是否符合 intent，这和你 Question Intent IR → Code Intent IR 思想非常接近，只不过你的目标域是 TableQA、Pandas 和表格 grounding。  arXiv 这可以帮助你把工作从“多加一个 LLM Reviewer”提升成更像 **neuro-symbolic semantic program auditing**。

还有一个非常重要的问题：不要根据官方 test 0–30 的结果反复改方法，再把这个 test 准确率作为论文结果。这属于 test leakage。开发阶段最好使用 WTQ/TabFact 的 validation/dev，或者从 train 固定切一份 development slice。官方 test 应该在方法和 prompt freeze 后再跑。如果你只是为了现在快速调代码，0–30 可以看，但必须标成 **DEBUG ONLY**，不能作为最终实验依据。

另外，你消息中直接贴出了 API key。建议现在就去平台作废并重新生成这个 key。下面的 Codex 指令我不会复述它，并且要求 Codex 只通过环境变量读取，禁止写进代码、日志或 Git。

下面这版我认为更适合直接交给 Codex，已经把研究目标、文件权限、实验纪律、实现顺序和停止条件都写清楚了。

编辑

Codex 执行指令：第二篇 TableQA 工作 - Multi-Granularity Semantic Logic Auditing

你现在负责我的第二篇 TableQA 论文代码项目。不要只给我方案或伪代码，要实际检查代码、实现方法、运行实验、查看日志、定位错误并迭代改进。

整个过程中，研究正确性、实验可复现性、文件安全性优先于追求短期准确率。

一、最高优先级文件权限约束

我有两个项目。

第一篇论文完整代码位于本地 Windows：

```
D:\master 1\CS\VS code project\rethinking\wangjie
```

这是已经完成的第一篇论文项目。

第一篇论文目录严格只读。

你可以：

- 阅读；
- 搜索；
- 理解；
- 比较代码；
- 将确实可以复用的普通源码复制到第二篇论文服务器项目。

你绝对不可以：

- 修改第一篇论文中的任何文件；
- 删除任何文件；
- 重命名文件；
- 格式化整个项目导致文件发生变化；
- 修改 requirements；
- 写入缓存；
- 创建实验输出；
- 创建临时文件；
- 修改 git 状态。

尤其不要复制 `.swp`、`.sw0`、`authorized_keys`、API key、`.env` 或其他可能包含凭据的文件。

第二篇论文当前在 VS Code SSH Linux 项目中。

只有当前 SSH Linux 第二篇论文 workspace 允许增删改文件。

每次开始工作前先执行并记录：

```
pwd  
git status
```

确认自己正在 Linux 第二篇项目后才允许写文件。

如果当前 SSH Codex 无法访问 Windows 的 `D:\...` 第一篇项目，不允许假装已经查看过。

此时：

1. 明确告诉我无法跨 SSH 访问该目录；
 2. 列出最希望从第一篇复制过来的文件/目录；
 3. 继续分析第二篇服务器上已经存在的代码；
 4. 不要因此修改第一篇项目。
-

二、研究背景

第一篇工作研究的是：

Breaking Self-Verification Bias in TableQA

核心思想是 Reasoner Agent、独立 Analyst Agent 和 Arbiter Agent，通过外部 Analyst 审计推理过程，缓解 LLM 对自己错误推理的 self-verification bias。

第二篇工作必须和第一篇形成自然延续，但不能只是换一个 prompt。

第二篇重点研究：

From Self-Verification Bias to Execution-Trust Bias

核心问题：

当 LLM 生成 Python 代码调用工具时：

代码成功执行 $\neq$ 代码正确实现了问题意图

Python interpreter 可以发现：

- Syntax Error
- Runtime Error
- KeyError
- TypeError
- ImportError

但是无法发现大量 **Silent Semantic Failure**：

- 问题要求 sum，代码执行了 mean；
- 问题要求 after 2010，代码写成 `>= 2010` ；
- 使用了错误列；
- 漏掉一个 filter；
- sort direction 错误；
- highest 被实现成 total；
- 选择了相似但错误的实体；
- 返回值类型与问题要求不一致。

这些代码完全可能成功运行，并给出一个错误数字。

将这种现象定义为候选研究概念：

Execution-Trust Bias

即工具增强 Agent 因为程序成功执行，而对程序结果产生不合理的过度信任。

三、第二篇论文真正的核心创新

核心贡献不能是：

- “让 LLM 写 Python”；
- “Python 报错以后重试”；
- “简单 self-reflection”；
- “文本跑 5 次、Python 跑 5 次然后多数投票”；
- “单纯动态选择文本还是 Python”。

核心创新必须聚焦：

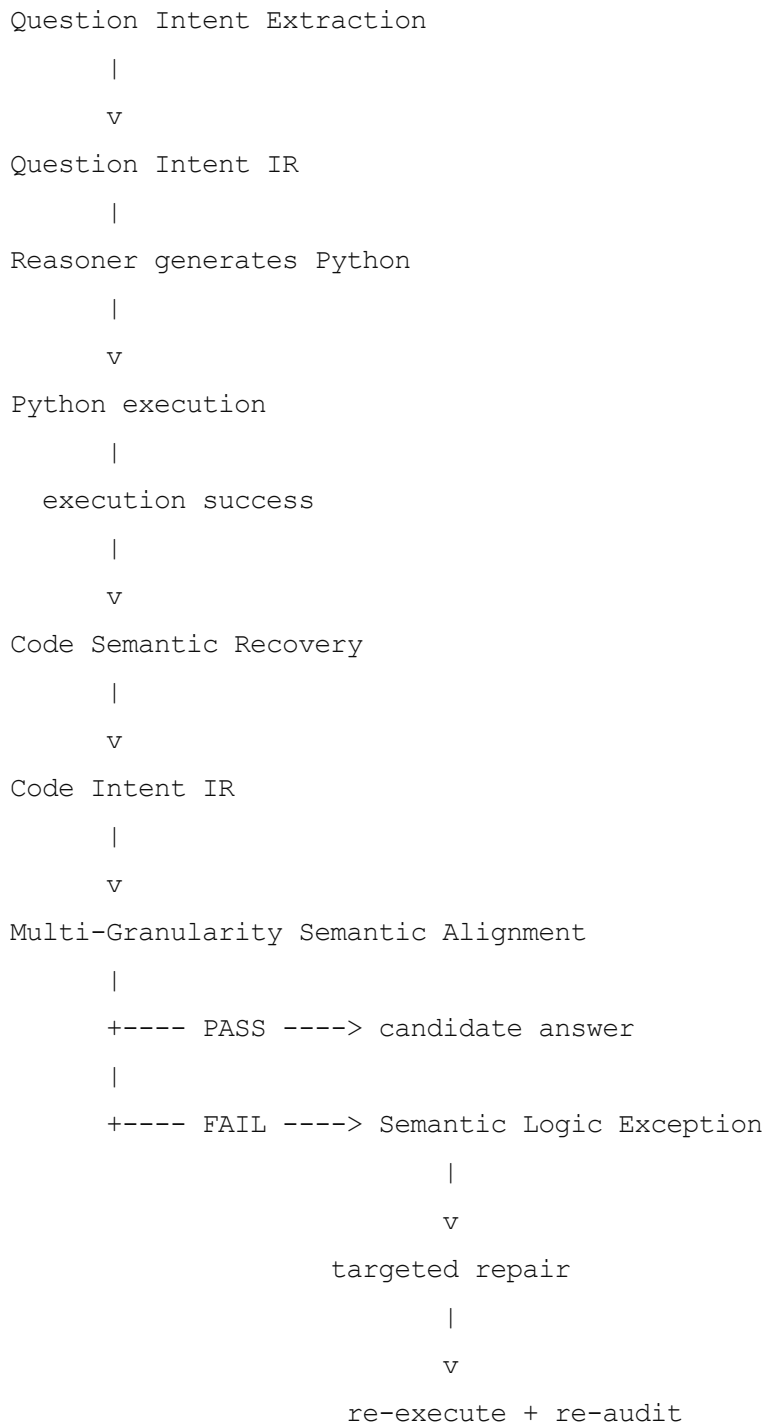
Successful-Execution Semantic Logic Auditing

即：

Python 已经成功运行以后，验证生成程序的实际语义是否忠实实现了原始 TableQA 问题。

目标流程：

Table + Question
|
v



注意：

审计重点发生在 execution success 之后。

四、必须实现真正的“三层多粒度语义审计”

这是论文最重要的结构之一，必须在代码架构、日志、实验和消融中真实存在，不能只是 prompt 里写三个标题。

Level 1: Global Intent Alignment

检查问题最终要求与程序最终返回目标是否匹配。

至少考虑：

- answer type:
 - number
 - entity
 - boolean
 - string
 - list
- answer cardinality:
 - single
 - multiple
- task:
 - lookup
 - aggregation
 - comparison
 - ranking
 - boolean verification
- expected return semantics。

例如：

Question:

Which country has the highest score?

Expected:

single entity

Code:

```
df["score"].sum()
```

Observed return:

number

应该触发：

```
SemanticLogicException
```

```
level = GLOBAL
```

```
type = ANSWER_TYPE_MISMATCH
```

Level 2: Operator Logic Alignment

将问题与 Python 分别转换为操作结构。

重点检查：

- filter
- select
- aggregation
- sum
- mean
- count
- max/min
- argmax/argmin
- comparison
- arithmetic
- difference
- groupby
- sort
- rank
- top-k
- temporal filtering
- join/merge（如果数据需要）
- negation
- boolean composition

例如：

Question:

What is the total sales in East after 2022?

Question Intent 应包括：

```
FILTER(region = East)
```

```
FILTER(year > 2022)
```

```
AGGREGATE(sum, sales)
```

如果代码实际只包含：

```
FILTER(region = East)
```

```
AGGREGATE(sum, sales)
```

必须检测：

```
MISSING_OPERATOR
```

```
missing = FILTER(year > 2022)
```

如果 Question 是 average，而 Code 是 sum：

```
OPERATOR_MISMATCH
```

```
expected = mean
```

```
actual = sum
```

Level 3: Parameter & Entity Alignment

检查具体参数。

包括：

- column name;
- table entity;
- cell value;
- numerical constant;
- date/year;
- comparison operator;
- string normalization;
- entity alias;
- filter value;
- target column;
- sort column。

例如：

```
Question:
```

```
after 2010
```

对应：

```
year > 2010
```

如果代码为：

```
year >= 2010
```

触发：

```
COMPARATOR_MISMATCH
```

```
expected = >
```

```
actual = >=
```

这一层不能全部依赖 LLM。

优先结合：

- table schema;
- column normalization;
- actual cell values;
- numeric/date parsing;
- string normalization;
- exact match;
- fuzzy matching;
- Python AST;
- runtime trace。

只有规则难以决定的问题，例如：

"East Region"

vs

"Eastern"

再调用 LLM 做语义判定。

五、统一 Intent IR

请设计稳定的 JSON/dataclass/Pydantic 数据结构。

Question Intent IR 示例：

```
{
  "answer_type": "number",
  "cardinality": "single",
  "target_columns": ["sales"],
  "filters": [
    {
      "column": "region",
      "op": "=",
      "value": "East"
    },
    {
      "column": "year",
      "op": ">",
      "value": 2022
    }
  ],
  "operations": [
    "filter",
    "filter",
```

```
    "aggregate"
  ],
  "aggregation": "sum",
  "ranking": null,
  "arithmetic": null
}
```

Code Intent IR 示例:

```
{
  "return_type": "number",
  "used_columns": [
    "region",
    "sales"
  ],
  "filters": [
    {
      "column": "region",
      "op": "=",
      "value": "East"
    }
  ],
  "operations": [
    "filter",
    "aggregate"
  ],
  "aggregation": "sum",
  "observed_output_type": "number"
}
```

AuditResult 至少包含:

```
{
  "passed": false,
  "semantic_exception": true,
  "global": {
    "passed": true,
    "errors": []
  },
  "operator": {
    "passed": false,
    "errors": [
      {
        "type": "missing_filter",
```

```
        "expected": "year > 2022"
    }
]
},
"parameter": {
    "passed": false,
    "errors": []
},
"repair_hint": "Add a strict filter year > 2022 before sum"
"confidence": 0.96
}
```

日志必须保存三个 level 的独立结果，以支持后续论文消融实验。

六、Code Intent IR 不能完全让 LLM 自己生成

实现 Hybrid Code Semantic Recovery。

A. Static Analysis

优先使用 Python `ast`。

尝试识别：

- dataframe column access;
- boolean comparison;
- filter;
- arithmetic operators;
- function calls;
- `.sum()` ;
- `.mean()` ;
- `.count()` ;
- `.max()` ;
- `.min()` ;
- `.idxmax()` ;
- `.idxmin()` ;
- `.sort_values()` ;
- `.nlargest()` ;
- `.nsmallest()` ;
- `.groupby()` ;
- `.merge()` ;

- `.iloc` / `.loc` ;
- comparison operators ;
- constants ;
- returned/final expression。

不要为了覆盖所有 Python 语法设计巨型 parser。

先覆盖 WTQ / TabFact 中最常见的 Pandas reasoning pattern。

B. Runtime Trace

代码成功执行后记录有用的 execution information，例如：

- intermediate dataframe shape ;
- selected columns ;
- number of rows before/after filtering ;
- final object type ;
- final scalar/list ;
- relevant intermediate values。

不能只保存 final result。

C. Table Grounding

使用真实 table schema 和 cell values 检查：

- 列是否存在；
- 实体是否真实出现；
- 数值/日期是否合法；
- 别名是否合理。

D. LLM Semantic Matching

仅用于规则难以处理的自然语言语义关系。

LLM 不能覆盖确定性的程序分析结果。

七、Semantic Logic Exception 与修复

区分两类失败。

Execution Failure

例如：

KeyError
SyntaxError
TypeError

走传统 execution error repair。

Semantic Failure

代码成功运行，但是 Audit 失败。

必须产生结构化：

Semantic Logic Exception

包括：

- level;
- error type;
- expected;
- actual;
- evidence;
- repair hint。

然后 Reasoner 只根据压缩后的 **semantic diagnostic** 修复代码。

不要把整个旧错误轨迹无限塞回模型。

默认：

max_semantic_repairs = 2

防止无限循环。

修复之后：

generate code
→ execute
→ semantic audit

必须重新完整执行。

不能因为修复后的代码可以运行，就跳过第二次 semantic audit。

八、先不要做复杂联合投票

不要直接实现：

Text x 5
Python x 5
majority vote

第一阶段优先证明论文最核心机制：

DeepSeek-V3.2 + Python

vs.

DeepSeek-V3.2 + Python + Multi-Granularity Semantic Audit

保持：

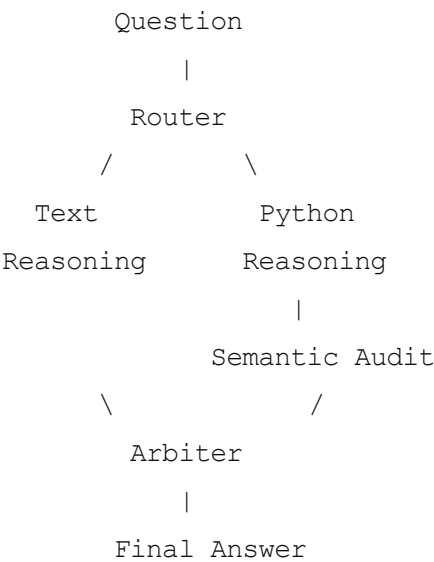
- 同一个模型；
- 同一批数据；
- 尽可能相同的 initial prompt；
- 相同 temperature；
- 相同 answer normalization；
- 相同 Python executor；
- 相同 evaluation script。

唯一核心变量是 Semantic Audit。

这样才能明确测量提升来自哪里。

九、核心方法证明以后，再增加 Adaptive Hybrid Reasoning

只有当 Python + Audit 已经稳定工作以后，再尝试第二阶段增强：



Router 可参考问题特征。

更偏 Python：

- arithmetic;
- aggregation;
- multi-row filtering;
- ranking;
- counting;
- numerical comparison;
- date calculation;
- multi-step deterministic operations。

更偏 Text：

- direct lookup;
- lexical ambiguity;
- entity interpretation;
- irregular schema;
- primarily semantic judgment。

Router 不确定时才双路运行：

```
1 Text candidate
+
1 Audited Python candidate
```

或者最多少量候选。

不要默认 10 次采样。

Routing 是辅助模块，不要让它掩盖 Semantic Audit 这个主创新。

十、实验组至少保留这些设置

最终代码应能通过配置运行：

- A. DeepSeek-V3.2 Text-only
- B. DeepSeek-V3.2 Python-only
- C. DeepSeek-V3.2 Python + Semantic Audit
- D. Adaptive Text/Python + Semantic Audit

其中：

B vs C 是最重要的核心机制实验。

D 是最终完整系统。

如果时间有限，优先把 B/C 做扎实。

十一、数据与实验纪律

数据集：

- WikiTableQuestions / WTQ
- TabFact

评估逻辑尽量复用第一篇论文已经验证过的数据 loader、normalization 和 evaluator，但只能复制到第二篇后修改。

非常重要：禁止 test leakage

不要根据官方 test 的 gold answer 反复调整：

- prompt;
 - 规则;
 - 阈值;
 - router;
- AST parser;
- 特殊 case。

开发和迭代优先使用：

1. 官方 dev / validation;
2. 如果项目没有现成 validation，则从 training split 创建固定 deterministic development slice。

例如：

```
development indices = 0..30
```

即 Python slice：

```
data[0:31]
```

固定 seed，并保存 sample IDs。

如果为了兼容现有项目必须临时跑官方 test 0..30：

必须把输出目录明确命名为：

```
debug_test_slice_DO_NOT_REPORT
```

它只能用于程序 smoke test。

不能基于它反复调参后再作为论文实验结果。

正式 test 必须在代码和 prompt freeze 后运行。

十二、用户当前要求的快速实验

开发阶段先进行小规模实验，避免立即烧大量 API。

在 WTQ 和 TabFact 的固定 development slice 上分别运行 indices：

```
0..30 inclusive
```

即 31 条样本。

首先运行：

```
Python baseline
```

保存：

- prediction;
- gold;
- generated Python;
- execution result;
- execution errors;
- raw model output;
- normalized prediction;
- correct/incorrect;
- token usage;
- API calls;
- latency。

然后运行：

```
Python + Semantic Audit
```

额外保存：

- Question Intent IR;

- Code Intent IR;
- Global audit;
- Operator audit;
- Parameter audit;
- Semantic Logic Exception;
- repair hint;
- repaired code;
- repaired result;
- final result。

生成错误 case 对比报告。

重点检查：

`baseline wrong + execution success`

这些就是 Silent Semantic Failure 候选。

分析 Auditor 是否能够：

1. 成功检测；
2. 正确定位；
3. 成功修复。

十三、不要只汇报最终 Accuracy

需要同时统计 Semantic Auditor 自己的指标。

在有 gold answer 的 development set 上：

如果：

```
execution success
AND
prediction != gold
```

将其标记为：

```
silent_failure_candidate
```

计算：

- silent semantic failure 数量；
- auditor detection precision；

- auditor detection recall;
- auditor F1;
- false positive rate:
 - 原代码正确但被 Auditor 错误拒绝;
- repair success rate;
- repair degradation rate:
 - 原本正确却被修错;
- Global-level exception 数量;
- Operator-level exception 数量;
- Parameter-level exception 数量;
- average API calls;
- average tokens;
- latency。

这是证明 Execution-Trust Bias 和 Semantic Audit 是否真正存在价值的重要实验。

十四、必须准备多粒度消融实验

代码结构必须允许容易关闭各层。

至少支持：

Full:

Global + Operator + Parameter

Ablation 1:

Global only

Ablation 2:

Global + Operator

Ablation 3:

Global + Operator + Parameter

Ablation 4:

LLM-only semantic audit

Ablation 5:

AST / grounding + LLM hybrid

优先验证：

Global
→ Global + Operator
→ Global + Operator + Parameter

准确率和 audit quality 是否逐级提升。

这样才能实证老师要求的“多粒度”。

十五、先编写人工 Semantic Bug 单元测试

在大量调用 API 前，先建立 synthetic semantic bug tests。

至少包含：

1. `sum` vs `mean`
2. `>` vs `>=`
3. wrong target column
4. missing filter
5. wrong entity
6. wrong sort direction
7. `highest` 被错误实现成 `sum`
8. `lowest` 被实现成 `max`
9. answer type mismatch
10. single vs multiple answer cardinality
11. wrong arithmetic sign
12. TabFact boolean polarity error

每个 case 都要标记应该在哪一层发现：

GLOBAL
OPERATOR
PARAMETER

运行：

pytest

确认基础 Auditor 工作后才开始大量 API 实验。

十六、API 调用规范

使用当前 Linux `work` 环境。

不要修改或破坏已经能正常工作的 Python 环境。

先检查：

```
conda activate work
python --version
pip list
```

只安装确实缺失的依赖。

需要 pip 安装时使用清华 PyPI 镜像。

不要随意升级整个环境。

当前项目 requirements 中已有固定依赖时优先遵守。

API 使用 OpenAI-compatible client。

模型：

DeepSeek-V3.2

API key 禁止硬编码。

只能读取环境变量，例如：

```
import os
import openai

client = openai.OpenAI(
    api_key=os.environ["PARATERA_API_KEY"],
    base_url=os.environ["PARATERA_BASE_URL"],
)
```

模型 ID 同样最好配置化：

```
MODEL_ID = os.getenv("MODEL_ID", "DeepSeek-V3.2")
```

禁止：

- 将 API key 写进 Python；
- 写入 YAML/JSON；
- 写入 README；
- 写进实验日志；
- commit 到 Git；
- print 完整 API key；
- 在异常 traceback 中泄露 key。

日志中如果发现 secret，立即 mask。

十七、API 工程实现要求

统一封装 API client。

至少支持：

- timeout;
- retry;
- exponential backoff;
- request error logging;
- usage statistics;
- model configuration;
- temperature configuration;
- deterministic experiment metadata;
- response caching。

Cache key 应考虑：

```
dataset
sample_id
model
method
prompt_version
question
table hash
```

Cache 不得包含 API key。

相同实验不要无意义重复付费调用。

十八、先理解现有项目，不要推倒重写

开始时检查第二篇 repo：

- README;
- requirements;
- dataset loader;
- prompts;
- evaluation;

- run scripts;
 - utils;
 - output;
 - existing agent;
- 现有 API wrapper。

同时只读检查第一篇代码。

找出可以安全复用的：

- WTQ loader;
- TabFact loader;
- evaluator;
- answer normalization;
- prompt formatting;
- logging;
- API wrapper;
- retry;
- experiment runner;
- result analysis;
- random seed logic。

复制以后在第二篇中独立修改。

尽量在现有结构基础上增加模块。

不要为了追求“代码漂亮”进行与论文方法无关的大规模 refactor。

十九、建议的代码模块

根据当前 repo 实际结构调整，不要求名字百分百一致，但逻辑最好解耦。

建议包括：

```
semantic_audit/  
    intent_schema.py  
    question_intent.py  
    code_intent.py  
    ast_analyzer.py  
    runtime_trace.py  
    grounding.py  
    auditor.py
```

```
exceptions.py
repair.py

routing/
  router.py

agents/
  reasoner.py
  analyst.py
  arbiter.py

evaluation/
  metrics.py
  audit_metrics.py
  error_analysis.py

scripts/
  run_wtq.py
  run_tabfact.py
```

不要创建重复功能。

先检查现有文件。

二十、每轮实验必须自动生成可分析结果

建议输出：

```
outputs/
  <experiment_name>/
    config.json
    predictions.jsonl
    summary.json
    audit_metrics.json
    cases/
    logs/
```

每条 JSONL 至少可追踪：

```
dataset
sample_id
question
gold
```


method
generated_code
execution_success
execution_error
execution_output
question_intent_ir
code_intent_ir
global_audit
operator_audit
parameter_audit
semantic_exception
repair_count
final_prediction
normalized_prediction
correct

错误分析要方便搜索：

baseline_wrong_audit_correct
baseline_wrong_audit_wrong
baseline_correct_audit_broke
silent_failure_detected
silent_failure_missed
false_positive_audit

二十一、每轮修改必须基于错误类别，而不是单条 hard-code

分析 0..30 或 development samples 时：

先聚类错误：

wrong column
missing filter
wrong comparator
wrong aggregation
answer-type mismatch
entity mismatch
code execution failure
auditor false positive
auditor false negative
repair failure

只有出现重复模式时才修改通用逻辑。

严禁写：

```
if sample_id == 17:  
    ...
```

严禁：

- 针对 gold answer hard-code；
- 针对某道 test question 写特殊条件；
- 根据 test label 调阈值。

每次改动后记录：

Problem:
Hypothesis:
Code change:
Expected effect:
Observed effect:
Regression:

二十二、迭代策略

不要无限盲目循环。

推荐：

Round 0

验证 environment、dataset、API、evaluator。

Round 1

跑 Python baseline。

Round 2

实现 Question Intent IR + Code Intent IR + 三层 Audit。

Round 3

加入 Semantic Logic Exception + targeted repair。

Round 4

分析 false positive / false negative，改进 AST / grounding / prompt。

Round 5

如果核心机制稳定，再尝试 Adaptive Hybrid Routing。

每次改进都必须重新跑同一固定 development slice 才能公平比较。

最多连续进行 5 个主要优化 cycle。

如果连续 2 个 cycle：

- overall metric 无提升；
- silent failure detection 无提升；
- 或提升只是通过明显增加 API 成本获得；

则停止盲目修改，输出当前最佳版本及剩余错误分析。

二十三、不要用 31 条数据得出论文结论

31 条只用于：

- smoke testing；
- debugging；
- architecture validation；
- direction check。

31 条里一个样本就会造成约 3.2 个百分点变化。

因此即使提升明显，也只能说：

`promising development signal`

不能直接声称显著优于某方法。

当小规模实验稳定后，再扩大 validation。

方法 freeze 后，最后才能运行官方 test。

二十四、重点观察的成功 case

优先寻找如下 case 写论文 case study：

Question:

What is the total sales of East after 2022?

Generated code:

```
filtered = df[df["region"] == "East"]  
answer = filtered["sales"].sum()
```

Python:

execution success

Baseline:

returns wrong answer

Question Intent IR:

```
region = East  
year > 2022  
aggregation = sum(sales)
```

Code Intent IR:

```
region = East  
aggregation = sum(sales)
```

Audit:

missing_filter: year > 2022

Semantic Logic Exception:

OPERATOR / PARAMETER

Repair:

```
add df["year"] > 2022
```

Re-execution:

success

Re-audit:

PASS

Final:

correct

这类 case 最能体现:

Syntax correctness

!=

二十五、科研故事不要被 routing 抢走

论文核心故事保持：

Self-Verification Bias

↓

第一篇：

independent reasoning audit

↓

工具增强推理

↓

Execution-Trust Bias

↓

successful code can still be semantically wrong

↓

Question Intent IR

↓

Code Intent IR

↓

Multi-Granularity Semantic Logic Auditing

↓

Semantic Logic Exception

↓

Targeted Repair

Adaptive routing、fallback、text/Python hybrid 都属于外围增强模块。

只有实验证明确实有帮助后再加入最终系统。

二十六、第一次开始执行时请按这个顺序

不要直接修改大量代码。

第一步先完成：

1. pwd + git status
2. inventory 第二篇 repo
3. 确认 work 环境
4. 找出 WTQ / TabFact loader

5. 找出 evaluator
6. 找出 API wrapper
7. 检查第一篇项目是否可读
8. 列出可以复用的第一篇文件
9. 给出当前 repo architecture summary
10. 给出最小修改方案

然后直接开始实现。

不要在给出计划之后停下来等待我逐步确认，除非：

- 需要访问第一篇 Windows 项目但完全无法访问；
- 缺少数据集；
- 缺少 API 环境变量；
- 即将执行破坏性操作；
- 或遇到无法自行解决的权限问题。

其余正常开发决策自行完成。

二十七、最终交付标准

完成当前 Codex 项目前，至少保证：

1. 第一篇论文项目没有发生任何修改；
2. 第二篇项目代码结构可运行；
3. API key 没有进入任何代码或日志；
4. WTQ baseline 可复现；
5. TabFact baseline 可复现；
6. Question Intent IR 可生成；
7. Code Intent IR 可生成；
8. 三层 Semantic Audit 已真实实现；
9. execution-success semantic failure 可以被检测；
10. Semantic Logic Exception 可以触发 targeted repair；
11. synthetic semantic bug tests 通过；
12. baseline vs audit 有固定 development comparison；
13. 保存完整结果与日志；
14. 有 Auditor precision/recall/FPR/repair success 等指标；
15. 有 Global / Operator / Parameter 多粒度分析；
16. 没有针对 sample/gold 的 hard-code；
17. 已总结失败案例；
18. 给出当前最佳 configuration；

19. 给出下一阶段扩大实验的建议；

20. 最终执行一次：

```
git status  
git diff --stat
```

并向我总结：

```
Files added:  
Files modified:  
Experiments run:  
Best configuration:  
WTQ result:  
TabFact result:  
Silent failures detected:  
Repairs successful:  
False positives:  
Known remaining problems:  
Recommended next experiment:
```

整个项目最重要的原则：

不要把“程序成功执行”当作“程序忠实实现了问题”。

第二篇工作的核心目标，就是系统性发现这种 **successful but semantically wrong** 的 TableQA 推理，并通过多粒度语义审计进行定位与修复。

如果按这个方案走，我认为最值得你优先做的并不是 routing，而是先把 `Python baseline` → 代码成功但答案错 → 三层 Auditor 定位 → repair → 正确 这条链真正跑通。只要 WTQ/TabFact 中能稳定找到一批这样的 silent failures，并且 Auditor 的 false positive 又控制得住，你第二篇论文的“现象—机制—实验”骨架就基本立住了。

来源

data的格式已经确定了，第一篇论文中就已经确定了不用你再修改了。

```
[
  {
    "table_id": "csv/203-csv/733.tsv",
    "table": {
      "header": [

```

展开

思考了 2m 5s

可以。我已经按你这次补充重新收紧了要求：**数据格式完全冻结，直接沿用第一篇；不再让 Codex 设计/修改数据处理；实验直接从现有 test set 抽取，先 5 条跑通，再逐步扩大到 10/20/30，必要时最高 40 条。**

这也和第一篇论文衔接得比较自然：第一篇已经把 Reasoner、独立 Analyst 和 Arbiter 作为核心框架，并在 WTQ、TabFact 上实验。1987806d-c35d-4de6-bad7-d10e6d4... 而第一篇的 Limitations 又明确指出当前 Analyst 尚未结合 Python/SQL 等 symbolic executor，并把 tool-assisted verification 作为后续方向。1987806d-c35d-4de6-bad7-d10e6d4... 所以第二篇现在可以更集中地做“**执行成功后的多粒度语义逻辑审计**”。

你直接把下面这段作为**最外层 Codex** 输入即可，详细规则让它继续读取我给你整理好的几个文件。

编辑

Codex 第二篇 TableQA 项目总指令

你现在开始执行我的第二篇 TableQA 论文项目。

我已经给你提供以下项目说明文件，请先完整阅读

`CODEX_MASTER_PROMPT.md`，然后阅读 `RESEARCH_METHOD_SPEC.md` 和 `EXPERIMENT_PROTOCOL.md`，之后再开始修改代码和运行实验。

你的任务不是只给我设计方案，而是实际完成：

检查现有项目 → 只读分析第一篇代码 → 复制可复用模块 → 在第二篇 Linux 项目实现 → 调用 DeepSeek-V3.2 API → 跑 WTQ/TabFact 实验 → 阅读日志和错误案例 → 修改通用方法 → 重跑 → 保留最优版本。

必须遵守以下最高优先级规则：

1. 第一篇论文项目

```
D:\master 1\CS\VS code project\rethinking\wangjie
```

严格只读，绝对不能修改任何文件。

2. 只有当前 VS Code SSH 连接的第二篇 Linux workspace 可以增删改文件。

3. Linux Python 环境已经创建，环境名为：

```
work
```

4. 数据格式已经由第一篇论文完全确定，禁止重新设计、重新预处理或修改数据 **schema**。

WTQ / TabFact 的：

- loader;
- table serialization;
- question-answer-id 对齐;
- normalization;
- evaluator;
- 已生成的 transposed/shuffled 数据;

全部优先复用第一篇已经验证过的代码。

第二篇只修改**推理**、**Python 执行**、**语义审计**和**修复机制**。

5. 第二篇核心问题是：

Execution-Trust Bias

即：

```
Python execution success != semantic correctness
```

真正需要解决的是 Python 已经成功运行以后，代码仍然可能因为错误列、遗漏过滤条件、错误 aggregation、错误 comparator、错误实体或错误 ranking 等产生 silent semantic failure。

6. 核心创新必须是：

Multi-Granularity Semantic Logic Auditing after Successful Execution

必须真实实现三个层次：

- Global Intent Alignment
- Operator Logic Alignment
- Parameter & Entity Alignment

三层必须分别有代码实现、日志输出、异常类型和消融开关，不能只存在于 prompt 描述中。

7. 推荐主流程：

```
Question → Question Intent IR → Python → Execution →  
Code Intent IR → Global/Operator/Parameter Audit →
```

Semantic Logic Exception → Targeted Repair → Re-execute
→ Re-audit

8. Code Intent IR 不能完全依赖 LLM。

优先组合：

- Python AST;
- Pandas operation analysis;
- runtime trace;
- table schema;
-真实 cell values;
- 数值/日期/string normalization;

LLM 只处理难以规则化的自然语言语义匹配。

9. 第一阶段不要做：

Text × 5 + Python × 5 + majority voting

先严格比较：

Baseline

DeepSeek-V3.2 + Python

与

Ours

DeepSeek-V3.2 + Python + Multi-Granularity Semantic
Audit

两者必须使用完全相同的 sample IDs、模型、evaluation、normalization 和 executor。

10. 测试直接使用当前已经确定好的 test set。

实验规模逐渐扩大：

5 → 10 → 20 → 30 → 最多 40

先用 5 条把完整流程跑通。

确认没有程序错误后，再逐渐扩大。

baseline 和 ours 每次必须使用完全相同的测试样本。

可以取连续 slice，也可以随机抽取，但随机时必须固定 seed，并把实际 sample IDs 保存下来，保证完全可复现。

11. 不允许针对单个测试题 hard-code。

禁止：

```
if sample_id == ...  
if question == ...  
if gold_answer == ...
```

修改只能针对日志中发现的通用错误类型。

12. 每轮实验必须保存完整结果，包括：

- question;
- gold;
- generated Python;

- execution result;
- execution error;
- Question Intent IR;
- Code Intent IR;
- Global Audit;
- Operator Audit;
- Parameter Audit;
- Semantic Logic Exception;
- repair hint;
- repaired code;
- final answer;
- correct/incorrect;
- API usage;
- latency。

13. 特别分析:

```
execution_success == true AND prediction != gold
```

这些是最重要的 silent semantic failure 候选。

需要分析 Auditor:

- 是否检测到;
- 在哪一个粒度检测到;
- 是否正确定位;
- repair 后是否恢复正确答案;
- 是否出现 false positive;
- 是否把原来正确的代码修错。

14. 必须提前支持多粒度消融:

- Global only
- Global + Operator
- Global + Operator + Parameter
- LLM-only Audit
- Hybrid AST/Grounding + LLM Audit

15. API:

模型:

```
DeepSeek-V3.2
```

OpenAI-compatible endpoint:

```
https://ai.paratera.com/v1/
```

API key 我会直接提供给当前 Codex/服务器环境。

不允许把 key commit 到 Git, 也不要完整 key 打印到日志。

16. 缺少 Python package 时使用清华 PyPI 镜像。

不要无理由升级整个 `work` 环境。

17. 开始工作后的第一步必须是:

```
pwd
```

```
git status
```

```
conda activate work
```

然后检查第二篇 repo 文件结构以及第一篇哪些模块可以安全复用。

18. 不要只给计划后停止。

除非遇到：

- 权限问题；
- 缺失 API key；
- 数据不存在；
- 无法访问第一篇项目且确实需要我手动复制文件；
- 或潜在破坏性操作；

否则你应该自行继续完成实现、实验和迭代。

19. 最多进行大约 5 个主要优化 cycle。

每轮根据实际日志中的错误类别改进通用方法，再重新跑同一组 samples。

如果连续两轮整体 accuracy、silent failure detection、repair success 和 false positive 都没有改善，就停止盲目修改并保留最佳版本。

20. 最终必须汇报：

- 第一篇项目是否保持完全只读；
- 复制了第一篇哪些代码；
- 第二篇新增/修改了哪些文件；
- WTQ 跑了哪些 sample IDs；
- TabFact 跑了哪些 sample IDs；
- Python baseline accuracy；
- Semantic Audit accuracy；
- execution-success-but-wrong 数量；
- semantic failures detected；
- repair success；
- false positives；
- repair degradation；
- Global/Operator/Parameter 分层统计；
- 当前最佳配置；
- 剩余失败案例；
- 下一步最值得做的实验。

完整的实现细节和实验约束以 `CODEX_MASTER_PROMPT.md` 为最高优先级规范。

现在请先读取这些文件和当前 repo，然后直接开始执行。

我给你整理好的文件如下。最方便的是直接下载 ZIP，把整个文件夹一起交给 Codex。

- [Codex 完整文件包 ZIP](#) — 推荐直接使用，里面已经包含所有说明文件和第一篇论文 PDF。
- [CODEX_MASTER_PROMPT.md](#) — 最重要的主指令，已经按“数据不动、test 5→40、多粒度审计、实际反复实验”全部重写。
- [RESEARCH_METHOD_SPEC.md](#) — 专门约束 Execution-Trust Bias、Question/Code Intent IR 和三级审计方法。
- [EXPERIMENT_PROTOCOL.md](#) — 专门规定 WTQ/TabFact 的 5→10→20→30→40 实验方式、同样本公平比较和日志指标。
- [API_SETUP_TEMPLATE.sh](#) — API 环境变量模板；真实 key 没有写入文件，你可以直接在 Codex 会话或服务器 shell 中提供，避免凭据意外进入 Git。
- [README_FOR_CODEX.md](#) — 告诉 Codex 这些材料该按什么顺序阅读。
- [第一篇论文 PDF](#) — 已经一起放入文件包。

这里我特意把之前那句“重新创建 development data”彻底删掉了。现在 Codex 得到的是很明确的原则：第一篇已经解决的数据、loader、evaluator 一律当基础设施复用；第二篇只碰 reasoning/tool/audit/repair。

另外，30 条的选择我也没有写死成“0~29”：Codex 可以截取，也可以固定随机 seed 抽 30 条，但 baseline 和你的方法必须跑完全相同的 ID。这样后面它告诉你“提升了多少”才有意义。

CODEX_MASTER_PROMPT.md
文档

RESEARCH_METHOD_SPEC.md
文档

EXPERIMENT_PROTOCOL.md
文档

还有 3 个

来源

